Oliver Greenwood, 7932Z

Whisperwise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks

Computer Science Tripos – Part II
Queens' College

2026

Declaration

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 7932Z, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. [The project required the approved use of [insert name of technology] and such use is acknowledged in the text at the relevant sections.] [I am content for my report to be made available to the students and staff of the University.]

Signed: Oliver Greenwood
Date: April 16, 2026

Proforma

Candidate number:	7932Z
Project Title:	**Whisperwise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks**
Examination:	**Computer Science Tripos – Part II, 2026**
Word Count:	⟨WORD COUNT⟩[1]
Code Line Count:	⟨LINE COUNT⟩[2]
Project Originator:	Oliver GReenwood
Project Supervisor:	Ian Lewis

Original Aims of the Project

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Work Completed

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Special Difficulties

None.

[1] This word count was computed using `texcount`.
[2] This code line count was computed using `cloc` (excluding autogenerated test output).

Contents

1 Introduction **1**

1.1 Project Overview & Contributions 2

1.2 Background Work 2

2 Preparation **3**

2.1 Project Overview 3

2.1.1 Simulation 3

2.1.2 Stations 3

2.1.3 Agents 3

2.2 Reinforcement Learning 4

2.2.1 Reward Function 4

2.2.2 Multi-Agent Proximal Policy Optimisation(MAPPO) 5

2.3 Protocols 7

2.3.1 No Communication 7

2.3.2 On-Policy Communication using Codebook 7

2.3.3 AI Mother Tongue 7

2.4 Requirements Analysis 12

2.4.1 Success Criteria 12

2.4.2 Extension 13

2.4.3 MoSCoW Analysis 13

2.5 System Design & Tool Selection 13

2.5.1 Software Development Model 13

2.5.2 Tool Selection & Justification 13

2.6 Starting Point 14

3 Implementation **15**

3.1 Implementation Overview . . . 15

3.1.1 System Layout . . . 15

3.1.2 Repository Overview . . . 16

3.1.3 Implementation Approach . . . 16

3.2 Environment . . . 16

3.2.1 Cross-Language Interoperability . . . 16

3.2.2 World Vectorisation . . . 16

3.2.3 Spatial Hashing . . . 17

3.2.4 Visualising the Environment . . . 17

3.3 MARL Infrastructure . . . 18

3.3.1 Actor and Critic Design . . . 18

3.3.2 Memory Management . . . 18

3.3.3 Generalised Advantage Estimation . . . 19

3.3.4 Communication Strategy Pattern . . . 19

3.3.5 MAPPO Algorithm . . . 20

3.4 AI Mother Tongue and Emergent Communication . . . 21

3.4.1 Autoencoder Architecture . . . 21

3.4.2 Offline Pre-training . . . 22

3.4.3 Reflection Strategies . . . 24

3.4.4 Architectural Divergence: On-Policy VQ-VAE . . . 25

3.5 Software Engineering, Reproducibility, and Observability . . . 25

3.5.1 Real-time System Monitoring . . . 25

3.5.2 Configuration Management . . . 26

3.5.3 Checkpointing . . . 26

3.6 Summary . . . 26

4 Evaluation **27**

4.1 Evaluation Setup . . . 27

4.2 System Performance . . . 27

4.2.1 Vectorised Environment Throughput . . . 27

4.2.2 Rollout Buffer Efficiency . . . 28

4.3 Offline Pre-training . . . 29

4.3.1 Reconstruction and Commitment Quality . . . 29

4.3.2 Initial Codebook Health . . . 29

4.4 Online MARL Task Performance . . . 29

4.4.1 Baseline Comparisons . . . 29

4.4.2 Bandwidth-Performance Trade-off . . . 29

4.5 Analysis of Emergent Linguistics . . . 29

4.5.1 Spontaneous Semantic Compression . . . 29

4.5.2 Intent and Semantic Mapping . . . 29

4.5.3 Decoding the HQ-VAE . . . 29

4.6 Architectural Ablation Studies . . . 29

4.6.1 Reflection Strategies - Are They Needed? . . . 29

4.6.2 VQ-VAE vs SQ-VAE . . . 29

5 Conclusions **30**

5.1 Achievements . . . 30

5.2 Lessons Learnt . . . 30

5.3 Future Work . . . 30

Bibliography **31**

A Proposal **32**

Chapter 1

Introduction

Since Google DeepMind released AlphaGo in 2016, there has been a steady increase in interest in the field of reinforcement learning. This has led to breakthroughs in areas of finance, trading, robotics and even video games. A clear next step for reinforcement learning is multi-agent reinforcement learning, where multiple agents are placed into the same environment and have to learn to interact with one another efficiently. This area of research has the potential to improve surgery robots, traffic optimisations and search-and-rescue operations.

However, there exists a range of challenges which make multi-agent reinforcement learning more difficult in practice, such as: non-stationary problem, credit assignment issues, and the one I will focus on in this project, communication problems. Unlike single-agent reinforcement learning, multi-agent reinforcement learning can use communication to improve the agent's knowledge of their environment and/or teach them the optimal next step. In Partially Observable Markov Decision Processes (POMDP), communication is required for agents to solve the problem.

Traditionally, communication is facilitated through every agent braodcasting raw continuous values to every other agent. While this is perfect for backpropagation, this scales poorly due to limited bandwidth between agents and is difficult for humans to interpret.

Replacing these continuous communication vectors with discrete tokens is highly desirable for both bandwidth and interoperability. However, learning a discrete language on-policy suffers from the "Joint Exploration Problem" where agents get stuck in a local optima because they learn to completely ignore the communication channel as it is initially solely noise. This local optima is named the "Communication Vaccum Equilibrium".

This dissertation focuses on the paper "AI Mother Tongue: Self-Emergent Communication in MARL via Endogenous Symbol Systems"[1]. In this paper, Liu et al. propose abandoning learning this discrete language during the reinforcement learning phase of training. Instead, they propose using a pre-trained discrete language model acting as a semantic bottleneck.

By using a Vector-Quantised Variational Autoencoder (VQ-VAE), continuous observations can be mapped to a finite set of discrete symbols. Liu et al. then showed that this autoencoder can be frozen during training and the agents can be taught semantic meaning through reflection strategies - custom loss functions that force agents to understand contextual meaning and intent of these symbols. This methodology allows communication protocols to evolve naturally without implementing hard biases ¬R.

Whilst the paper suggests an interesting novel idea, the paper only evaluates this on a simple niche idea. To ascertain whether this architecture is effective and robut enough for practical use, it requires rigourous evaluation within a more complex environment.

1.1 Project Overview & Contributions

In this project, I:

1. **Scale the Algorithm** The AI Mother Tongue architecture will be integrated into a highly efficient Multi-Agent Proximal Policy Optimisation (MAPPO) framework to test the limits of the architecture.
2. **Generative Language Extension** The project will implement the VQ-VAE approach as well as experiment with other generative language models suggested by the paper such as Stochastically-Quantised Variational Autoencoder (SQ-VAE) and Hierarchically-Quantised Variational Autoencoder (HQ-VAE) to determine if probabilistic or multiple codebooks improve coordination.
3. **Emperical Benchmarking** The AI Mother Tongue architecture will be quantitivly compared to other communication methods such as no communication and continuous communication methods.

1.2 Background Work

Previous work:

Obvious direct related work:
AI Mother Tongue - implementing this VQ-VAE idea into basic 2 agent tasks
VQ-VAE - Paper from a few years ago
TRPO -¿ PPO -¿ MAPPO
Reinforcement learning paper
Current communication in RL

Other important works:
VQ-VAE for compression - see info theory
Some form of analysis of communication using information theory

Chapter 2

Preparation

Well-motivated project with success criteria well-justified. Challenging and well-presented background covering Comp Sci topics beyond Part IB. Good requirements analysis, justified selection of suitable tools, good engineering approach.

2.1 Project Overview

2.1.1 Simulation

To evaluate communication, we investigate through the following cooperative game.

There is an MxN board. The game lasts for T timesteps and at every timestep, every agent simultaneously does one action. The goal of the game is for the agents to cooperate and complete a required recipe.

2.1.2 Stations

Stations are dotted randomly across the map and are immovable throughout the course of the game. Each station has an assigned ingredient. The station can either be of type 'pick up' or 'drop off'. A 'pick up' station only allows the agents to take ingredients from them. A 'drop off' station only allows the agents to give the station the assigned ingredient.

2.1.3 Agents

Each agent is two sets of ingredients - an input set and an output set. Their input set determines which ingredients the agents need to collect. Once they have all of their ingredients, they get removed from their inventory and replaced with their output ingredients. The agents can collect ingredients from 'pick up' stations or other agents. The agents can drop off their ingredients at 'drop off' stations.

One important feature of this game is the agents have limited range of sight and can only view the nearest entity at any given timestep. This converts this game from a Markov Desicion Process to a Partially Observable Markov Decision Process. The agents must work together to discover the required stations and use them to complete recipes.

At every timestep, every agents takes a single action. Agents have five possible actions: move north, move south, move west, move east and interact. The first four of which move the agent one step in the given direction. If agent A executes the 'Interact' action:

1. If visible agent B has a required item, place in A's inventory, and remove from B's inventory.
2. If visible 'pick up' station has a required item, place in A's inventory.
3. If visible 'drop off' station wants an item we have, remove from A's inventory.

2.2 Reinforcement Learning

Reinforcement Learning (RL) is different to classical supervised and unsupervised learning as we do not have all the data before training. We must build environments for agents, who take actions and recieve feedback from these actions. The agents must learn to maximise the total reward collected over time.

Classic RL is modelled through Markov Decision Processes (MDPs), but to investigate communication, we must limit the information every agent has so they must communicate missing information with one another. Therefore, we will model this RL as a Partially Observable Markov Decision Process (POMDP).

A POMDP is formally defined as $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle$. Where:

- $\mathcal{S}$ is the set of all possible states of the environment.
- $\mathcal{A}$ is the action space
- $\mathcal{P}$ is the transition probability function $P(s'|s, a)$, representing the probability that taking action a in state s will lead to state s'
- $\mathcal{R}$ is the reward function
- Ω is the set of all possible local observations
- $\mathcal{O}$ is the observation function $P(o|s, a)$, representing the agent seeing observation o given state s and previous action was a
- γ is the discount factor

2.2.1 Reward Function

Liu et al.[1] suggests for these more complex games, a dense reward system should be implemented, giving agents to gain useful feedback on every step.

Each agent i:

$$r_t^{(i)} = r_{\text{time}} + r_{\text{move}} + r_{\text{novel}} + r_{\text{interact}} + r_{\text{share}}$$

Where:

$$r_{\text{move}} = \begin{cases} -0.02, & \text{if } a_t^{(i)} \in \{\text{Up}, \text{Down}, \text{Left}, \text{Right}\} \text{ and } \ell_{t+1}^{(i)} = \ell_t^{(i)}, \\ 0, & \text{otherwise.} \end{cases}$$

$$r_{\text{novel}} = 0.002\, \mathbb{I}_{\text{new}}(\ell_{t+1}^{(i)})$$

$$r_{\text{interact}} = \begin{cases} 0.5\, \mathbb{I}_{\text{found}}^{(i)} + 0.7\, \mathbb{I}_{\text{basic}}^{(i)} + 1.0\, \mathbb{I}_{\text{combine}}^{(i)}, & \text{if } a_t^{(i)} = \text{Interact}, \\ 0, & \text{otherwise.} \end{cases}$$

$$\text{FOUND_REWARD} = 0.5, \quad \text{BASIC_REWARD} = 0.7, \quad \text{COMBINE_REWARD} = 1.0$$

$$\bar{r}_t = \frac{1}{N} \sum_{j=1}^{N} \tilde{r}_t^{(j)}$$

$$r_t^{(i,\text{final})} = (1 - \alpha)\, \tilde{r}_t^{(i)} + \alpha\, \bar{r}_t$$

Idea behind rewards

Whats important spare reward problem - how we mitigate credit assignment problem - how we mitigate non stationary problem

Joint Exploration Dilemma Communication Vacuum Equilibrium Semantic Drift

Normalisation of rewards

2.2.2 Multi-Agent Proximal Policy Optimisation(MAPPO)

TODO Citations for MARL problems

The focus of this project is on communication in multi-agent reinforcement learning. However, MARL has many challenges which we are likely to uncover such as the non-stationary problem, credit assignment problem and long training times. The non-stationary problem is when multiple agents learn simultaneously creating an unpredicatably changing environment from an individual agent's perspective. To mitigate these problem, so we can focus our investigation on the communication aspect, we will adjust the original AI Mother Tongue[1] algorithm by replacing their chosen reinforcement learning algorithm of REINFORCE[2] to one which focuses on stability. We will use Multi-agent Proximal Policy Optimisation (MAPPO)[3] - an extension of an approximation of trust region policy optimisation (TRPO) for multiple agents. I have chosen MAPPO in hopes to reduce training times as it only requires a single critic and all agents can share the majority of paramaters, and the global critic can overcome the non-stationary problem and credit assignment problem as argued in [3].

MAPPO optimises two main components. Each agent's Policy (Actor) π_{θ_i} and the Value Function (Critic) V_ϕ.

Policy Objective (Actor)

The goal of the policy is to maximise the clipped surrogate objective, preventing drastic changes to the policy from single timesteps. For every agent i, define the probability ratio $r_i(\theta)$ as:

$$r_i(\theta) = \frac{\pi_\theta(a_i|o_i)}{\pi_{\theta_{old}}(a_i|o_i)} \tag{2.1}$$

The MAPPO objective function is:

$$L(\theta) = \mathbb{E}_i \left[\min \left(r_i(\theta)\hat{A}_i,\ \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i \right) \right] \tag{2.2}$$

Where:

- $\hat{A}_i$ is the Advantage Estimation
- ϵ is a hyperparameter that limits the policy update

Value Function (Critic)

Unlike standard Proximal Policy Optimisation (PPO)[4], the Value Function is not per-agent, but per-world. It has access to global information. The goal of the critic is minimise the Mean Squared Error between its own prediction and the discounted returns R_t.

$$L(\phi) = \mathbb{E}_t \left[(V_\phi(S_t) - R_t)^2 \right] \tag{2.3}$$

Generalised Advantage Estimation (GAE)

MAPPO uses Generalised Advantage Estimation[5] to balance bias and variance. To calculate, we first find the Temporal Difference (TD) Residual (representing the error in our current estimate of the state value):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{2.4}$$

We then take an exponentially weighted moving average of our estimated advantage over different time horizons.

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{2.5}$$

2.3 Protocols

Making my software re-usable and flexible by making communication protocols interchangeable will allow for comparisons between the protocols.

2.3.1 No Communication

To provide a comparable baseline, we will implement a protocol where agents do not broadcast any information to one another.

2.3.2 On-Policy Communication using Codebook

This is a common (TODO: JUSTIFY) modern methodology to facilitate communication between agents. A simple continuous vector being broadcast from each agent adds a lot of noise to the agents' policy. To reduce this noise, we discretise the continuous vector into a set number of codes. (TODO: Maths, citations)

One issue with this method alone is it is susceptible to codebook collapse. This is when the agents' policy only ever output a small subset of these codes. To mitigate this, we will implement dead code revival. (TODO: Maths, citation)

2.3.3 AI Mother Tongue

Vector-Quantised Variational Autoencoder (VQ-VAE)

Initially introduced by van den Oord et al. [6] and re-explained by Liu et al. [1], VQ-VAE is a generative model that differs from the standard Variational Autoencoder (VAE) by using a quantised latent space.

1. **Components** It is built from three components:

   (a) **Encoder**: Maps input x to a continuous latent representation z_e

   $$z_e = Enc_{\theta_E}(x) \tag{2.6}$$

   (b) **Quantiser**: Quantises the continuous latent vector z_e into a discrete symbol $z_q \in C$ where C is a finite set of vectors.

   $$k^* = \arg \min_{k \in \{1,\ldots,K\}} \|\mathbf{z}_e - \mathbf{e}_k\|_2^2. \tag{2.7}$$

   $$\mathbf{z}_q = \mathbf{e}_{k^*}. \tag{2.8}$$

   $$\mathbf{z}_q = \arg \min_{\mathbf{e}_k \in \mathcal{C}} \|\mathbf{z}_e - \mathbf{e}_k\|_2^2 \tag{2.9}$$

(c) **Decoder**: Reconstructs the input $\hat{x}$ from the discrete symbol z_q.

$$\hat{x} = Dec_{\theta_D}(z_q) \tag{2.10}$$

2. **Straight-Through Estimator (STE)**

   An issue of VQ-VAEs, is the argmin operator is non-differentiable. If we were to backpropagate classically, gradients would stop at the quantisation step. To solve this, VQ-VAE uses the Straight-Through Estimator where we simply copy the gradients from the decoder input z_q directly to the encoder output z_e.

3. **Loss Components**

   VQ-VAE Loss Function

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{codebook}} + \beta\mathcal{L}_{\text{commit}}, \tag{2.11}$$

   where $sg[\cdot]$ denotes the stop-gradient operator, which prevents gradients flowing through its argument in backpropagation. The components are:

   (a) *Reconstruction loss*:

$$\mathcal{L}_{\text{recon}} = \|x - \hat{x}\|_2^2, \tag{2.12}$$

   Defined as the MSE loss between the input x and the output $\hat{x}$

   (b) *Codebook loss*:

$$\mathcal{L}_{\text{codebook}} = \|\text{sg}[z_e(x)] - e\|_2^2, \tag{2.13}$$

   moves codebook vectors e towards encoder outputs $z_e(x)$.

   (c) *Commitment loss*:

$$\mathcal{L}_{\text{commit}} = \|z_q(x) - \text{sg}[z_e(x)]\|_2^2 \tag{2.14}$$

   commits encoder outputs to the nearest codebook entry, weighted by $\beta = 0.25$.

Stochastically-Quantised Variational Autoencoder (SQ-VAE)

As previously discussed, one major issue of using codebooks is codebook collapse. Although this is less prominent in vector-quantised variational autoencoder method implemented in the previous section than the basic version of the on-policy codebook generation (no dead code revival), it still exists due to the 'hacky' straight-through estimator leading to an approximation of the true gradient. To fix this issue, Takida et al. [7] propose Stochastically-Quantised Variational Autoencoder (SQ-VAE), a variational Bayes framework for learning VQ-VAE components.

Instead of using the argmin operator, we calculate a categorical probability distribution over the codebook. The probability of picking code k is based on a Gaussian distribution with a learnable paramater σ.

$$q(z_q = e_k|z_e) \propto \exp\left(-\frac{\|z_e - e_k\|_2^2}{2\sigma^2}\right) \tag{2.15}$$

We apply self-annealing whilst learning σ^2. Early during training, the probability distribution is wide which helps explore the entire codebook. Later on, σ^2 naturally shrinks, allowing the model to make hard choices. To make this differentiable, we use the Gumbel-Softmax trick. In

the forward pass, we force the output to be a strict one-hot vector. In the backward pass, we act as if the output was continuous, following this formula:

$$p_k = \frac{\exp(\frac{(logits)_k + g_k}{\tau})}{\sum_{j=1}^{K} \exp(\frac{(logits)_j + g_j}{\tau})} \tag{2.16}$$

where:

$$\tau = \max(\tau_{min}, \tau_{old} * annealing_rate) \tag{2.17}$$

However, this method still uses Euclidian distance, but in tasks such as NLP and audio, semantic meaning of a feature is often entirely captured by the direction of the vector, not the magnitude. To improve this method for our use case, we can implement an extension of this model von Mises-Fisher Stochastically-Quantised Variational Autoencoder (vMF SQ-VAE). vMF SQ-VAE uses cosine similarity instead of Euclidian distance. This is done by first normalising the variables onto a unit sphere.

$$z_{norm} = \frac{z_e}{\|z_e\|_2}, \quad e_{norm,k} = \frac{e_k}{\|e_k\|_2} \tag{2.18}$$

Then using the Gumbel-Softmax trick again but with cosine similarity instead of Euclidian distance.

$$q(z_q = e_k | z_e) = \frac{\exp(\kappa \cdot z_{norm}^T e_{norm,k})}{\sum_j \exp(\kappa \cdot z_{norm}^T e_{norm,j})} \tag{2.19}$$

Residually-Quantised Variational Autoencoder (RQ-VAE)

For a model to capture finer, more complex details, a larger vocabulary is required. Simply increasing the vocabulary size of the codebook can increase training time massively due to struggling to fit the entire embedding matrix in memory and the model struggling to explore the entire codebook. To prevent this, but still gain a larger vocabulary, we can use Residually-Quantised Variational Autoencoder (RQ-VAE).

We can apply this method to all of the pre-mentioned Variational Autoencoder methods (VQ-VAE, SQ-VAE). We calculate z_q from z_e as usual for the respective method, then we calculate a new z_e^1 from these:

$$z_e^{'} = z_e - z_q \tag{2.20}$$

This new $z_e^{'}$ can be used to calculate another z_q.

$$z_e^{(n)} = z_e^{(n-1)} - z_q^{(n-1)} \tag{2.21}$$

This process can be repeated any number of times (RQ Levels). Once this process has been repeated the chosen number of times, a final $\hat{z_q}$ can be calculated by summing all previous quantised vectors.

$$\hat{z_q} = \sum_{n=1}^{N} z_q^{(n)} \tag{2.22}$$

This produces an effective vocabulary size of:

$$\text{Effective Vocab Size} = \text{Original Vocab Size}^{\text{RQ Levels}} \tag{2.23}$$

Hierarchically-Quantised Variational Autoencoder (HQ-VAE)

Liu et al. suggested implementing Hierarchically-Quansited Variational Autoencoder (HQ-VAE) [8]. The idea behind this is splitting the communication into high-level strategies and low-level strategies, where the low-level strategy is conditioned on the high-level strategy.

We first extract common features through a shared encoder:

$$h_{\text{shared}} = Enc_{\theta_E}(x) \tag{2.24}$$

The top level, responsible for macro-level strategies, projects these shared features to a vector:

$$z_{\text{top}} = MLP_{\text{top}}(h_{\text{shared}}) \tag{2.25}$$

The bottom level, responsible for micro-level strategies, projects these shared features conditioned on the top level to a vector:

$$z_{\text{bottom}} = MLP_{\text{bottom}}([h_{\text{shared}}, z_{\text{top}}]) \tag{2.26}$$

We quantise both of these vectors using one of our aformentioned methods (VQ-VAE, SQ-VAE).

$$z_{q,top} = \text{Quantiser}(z_{top}) \tag{2.27}$$

$$z_{q,bottom} = \text{Quantiser}(z_{bottom}) \tag{2.28}$$

Finally, we concatenate both of these vectors:

$$z_q = [z_{q,top}, z_{q,bottom}] \tag{2.29}$$

Pretraining via Imitation Learning

To further reduce the training time required, Liu et al. [1] suggests initially training both the actor and critic models with supervised learning. Required observations, optimal actions and their associated advantages can be precollected from an expert agent.

1. **Expert Agent**: In order to do supervised learning on our actor and critic models, we require optimal data. In our game, creating an optimal agent is intractable as we must decide the most optimal route to travel to multiple stations. This is the travelling salesperson problem, a famous NP-Complete problem. Instead, we create a greedy expert agent, who always travels to the nearest station they require. Further optimistations can be done by PPO during RL.

2. **Supervised Learning**: To simulate communication, we send the location of the agent's own nearest current target in its communication slot encoded via a learned codebook lookup from spatial binning. We then use the actor to output actions logits and use cross-entropy to calculate loss:

$$\mathcal{L}_{\text{actor}} = -\log \pi(a_b^* | s_b, c_b) \tag{2.30}$$

   We also use the critic to predict the returns and use MSE to calculate loss:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t \left[(V_\phi(S_t) - R_t)^2 \right] \tag{2.31}$$

Reflection Strategies and Auxillery Losses

A large emphasis by Liu et al. [1] is the importance of reflection strategies. They introduce additional loss terms during the RL phase, aimed at fostering more xomplex learning and communication beyond simple reward maximisation. These reflection losses are defined over *sender* and *reciever* contexts constructed from observations and communication symbols. Due to the complexity of my environment, I have had to extend these auxillery losses to be more task-specific.

For each agent, the **sender context** is defined by the concatenation of its local observations and the communication vector it emits. The **reciever context** concatenates the agents local observations and all communication vectors recieved from other agents.

1. **Predicted Return Loss** Given **sender context**, we train an auxilliary head to predict the true return R_t collected from our GAE advantage estimation. Loss is defined by:

$$\mathcal{L}_{\text{return}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{R}(c_i^{\text{send}}) - R_i \right)^2, \tag{2.32}$$

   This mirrors Liu et al.'s value-prediction reflection, aligning communication with future rewards.

2. **Predicted Critic Loss** Given the **sender context** again, we train another auxillery head to predict the critic's current values estimate.

$$\mathcal{L}_{\text{critic}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{V}(c_i^{\text{send}}) - V_\phi(S_t) \right)^2, \tag{2.33}$$

3. **Intent Loss** The intent loss described in the paper models how agents messages influence partners' behaviour and rewards, however, this is not suitable for our needs. Instead, we will will predict their own target using **sender context** and predict other agent's target using **reciever context** (explicitly promoting a Theory-of-Mind style understanding of partners' intentions).

   These losses summed will be our intent loss. We define:

$$\mathcal{L}_{\text{sender-goal}} = \frac{1}{N} \sum_{i=1}^{N} \eta_i \left\| \hat{\tau}^{\text{send}}(c_i^{\text{send}}) - \tau_i \right\|_2^2, \tag{2.34}$$

$$\mathcal{L}_{\text{receiver-goal}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{\substack{j=1 \\ j \neq i}}^{N} \eta_j \left\| \hat{\tau}^{\text{recv}}(c_i^{\text{recv}}, j) - \tau_j \right\|_2^2, \tag{2.35}$$

   where

   - $\hat{\tau}^{\text{send}}$ predicts the sender's own target from its context
   - $\hat{\tau}^{\text{recv}}$ predicts another agent's target from the receiver context
   - $$\eta_i = \begin{cases} 1, & \text{if agent } i \text{ has a valid target,} \\ 0, & \text{otherwise.} \end{cases}$$

   The total intent loss is then:

$$\mathcal{L}_{\text{intent}} = \mathcal{L}_{\text{sender-goal}} + \mathcal{L}_{\text{receiver-goal}}. \tag{2.36}$$

The total reflection loss will then be:

$$\mathcal{L}_{\text{refl}} = \lambda \left(\mathcal{L}_{\text{return}} + \mathcal{L}_{\text{critic}} + \mathcal{L}_{\text{intent}} \right), \tag{2.37}$$

with a small coefficient λ (e.g. 0.01) so that reflection regularises, rather than dominates, the RL objective.

2.4 Requirements Analysis

2.4.1 Success Criteria

1. Design and implement a simulation to train and demo multi-agent interactions. The simulation should be able to have single runs visualised. The simulation should control the state of every agent and collect data.
2. Implement a functional MAPPO-based multi-agent reinforcement learning (MARL) system capable of learning, given a simulation program with set methods.
3. Develop a communication protocol that allows agents to transmit raw data between each other.
4. Similarly, develop another communication protocol that uses a VQ-VAE to transmit vectors between agents.
5. The training and evaluation pipeline should be able to train and evaluate models automatically and allow interchangeably using both communication protocols.

2.4.2 Extension

- Implement a third communication protocol using a hierarchically quantised variational autoencoder (HQ-VAE). This method was suggested by the AI Mother Tongue paper as a possible future work.
- Add hyperparameter tuning for my discrete communication protocol. This can involve the language size, quantisation levels, and message frequency to optimise agent coordination and learning efficiency.
- Add a new task where agents must solve for objectives other than an assembly line (e.g., coordinated navigation, resource gathering, or adversarial avoidance), enabling analysis of communication protocol effectiveness in more diverse MARL settings.

2.4.3 MoSCoW Analysis

Table 2.1: MoSCoW Analysis of project requirements and associated risks.

Priority	Feature	Risk
Must Have		

2.5 System Design & Tool Selection

2.5.1 Software Development Model

2.5.2 Tool Selection & Justification

Python was chosen as the programming language for the controller, primarily due to its excellent library support for machine learning and data analysis (including but not limited to NumPy ¬R, Pandas ¬R, Matplotlib ¬R and Jupyter Notebooks ¬R). PyTorch ¬R was chosen to develop the models due to its high performance and imperative execution model, which facilitates rapid iterative prototyping. However, Python is not a performance language and, as of development ¬R, has a GIL ¬R meaning executing simulations of my environment must be effectively done serially, bottlenecking my performance heavily. I chose Rust to develop my environment for its high performance and memory safety allowing for rapid development.

To streamline my package management and environment integration to the controller, I used a pyproject.toml file. This file handled installing packages from external sources and also building my rust environment program to allow my Python controller to access the environment.

For version control, I used GitHub to store my git repository which included both my program and my dissertation. I also regularly updated the version of code on my laptop as an extra backup.

2.6 Starting Point

I have extensive prior experience using Python. My internship over the previous summer introduced me to the Rust programming language and using the Py03 library to work with both Rust and Python. I had no prior experience with PyTorch, however I understood the fundamentals of neural networks from Cambridge courses and self instructed exercises such as creating a neural network from scratch to solve XOR and using genetic algorithms on the game Snake.

Chapter 3

Implementation

Contribution to the field, with genuine potential for impact outside the tripos. Challenging goals and substantial deliverables, all methods and tools deployed expertly. Original techniques or methodologies going beyond what was previously known. Presentation is clear and concise throughout, with creative use of figures or diagrams. Excellent repository overview, giving clear insight into project structure.

Summary

3.1 Implementation Overview

3.1.1 System Layout

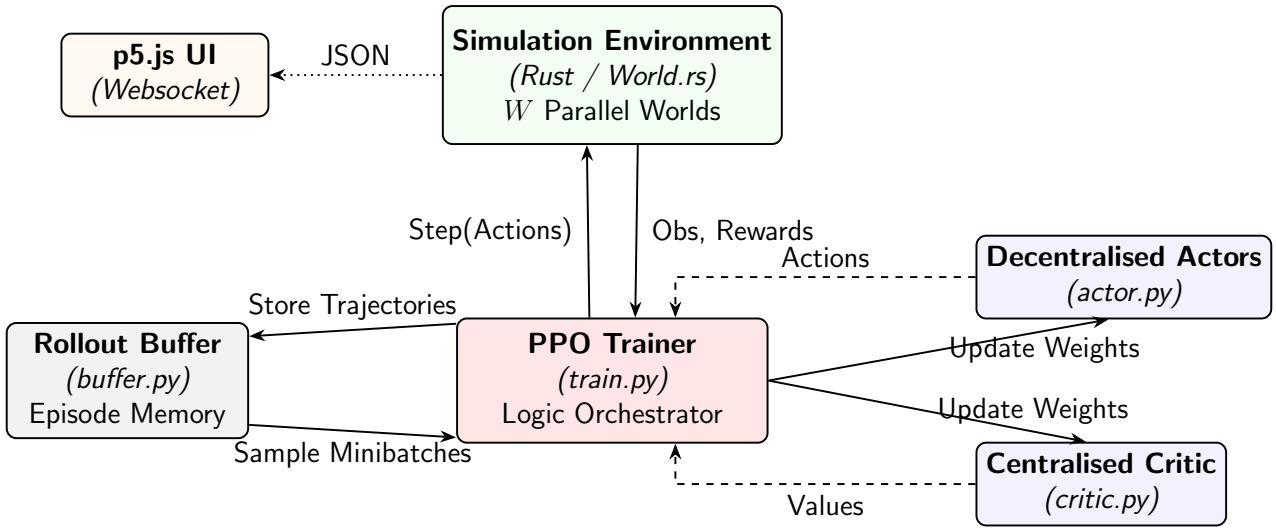

Figure 3.1: System Architecture Diagram: The PPO Trainer orchestrates the rollout phase with the Rust environment and the optimization phase using data sampled from the Rollout Buffer.

3.1.2 Repository Overview

3.1.3 Implementation Approach

Unit Testing

Documentation

3.2 Environment

TODO: Discuss env maybe?

Reinforcement learning requires millions of datapoints to produce sufficiently usable policy. In order to collect this data efficiently, a vectorised environment with efficient movement of data between the RL script and the Rust simulation is designed.

3.2.1 Cross-Language Interoperability

To allow the Rust simulation to be callable from the Python RL training scripts, PyO3 [9] bindings are developed. A fundamental bottleneck of Python<=3.13 is the Global Interpreter Lock (GIL) ¬R - a mutex that protects access to Python objects by preventing multiple threads from executing Python code simultaneously. The Rust library `PyO3` generates C-compatible Foreign Function Interface (FFI) bindings that can be called from our Python scripts. This FFI is compiled using setuptools-rust, a plugin for the Python library setuptools library [10] that allows for the compilation of Rust extensions. This toolchain is defined within the `pyproject.toml` configuration enabling all building into a single installation command.

For interacting with the simulation from Python, a `SimWrapper` class is developed following the Adapter Design Pattern [11]. This wrapper isolates the high-level reinforcement learning algorithm from the low-level foreign function interface mechanics. PyTorch works in high dimensional matrices and batched state representation, however it is considerably computationally faster to pass flat arrays to the Rust simulation to exploit CPU cache locality and parallel throughput. To take a simulation step, the `SimWrapper` class reshapes the data and passes it to the Rust simulation. On return from the Rust simulation, PyO3 provides an option to return a numpy[12] array. This is effectivly a pointer to a contiguous section of memory, allowing reshaping back to PyTorch's expected tensor shape in O(1) time (zero copying required).

3.2.2 World Vectorisation

Due to Python's GIL true parallelisation of CPU tasks is not possible. To circumvent this, we offloaded the parallelisation logic to the Rust simulation and step through multiple independent instances of the world concurrently. A single `Simulation` class is created to hold a vector of `World` states currently being simulated. According the Amdahl's Law ¬R, the theoretical speedup of a program is bottlenecked by its sequential fraction. The Rust library rayon ¬R is used to parallise the computation of each world to minimise this sequential overhead. This computation includes:

- Applying actions chosen from the policy
- Collecting rewards for feedback for the policy
- Collecting each agent's observations and each world's global observations for input to the actor and critic networks

The number of worlds in parallel is decided through empirical profiling. For my computer, worlds_parralised was optimally set to 32, however, this can easily be modified in the configuration files.

3.2.3 Spatial Hashing

In the environment, the closest entity (agent or station) to each agent is part of each agent's observations. A naive implementation would be checking every entity's euclidian distance to every agent. This scales with $O(N^2)$, where N is the number of entities. To resolve this, a `GridMap` struct is constructed using spatial hashing. Every world is represented as a `Vec<Vec<GridCell>>`, where agents register their presence upon movement, and stations register on initialisation. Using this data structure, we can convert our $O(N^2)$ time complexity to $O(1)$. On top of this functionality, we can also easily calculate an exploration reward. Each `GridCell` contains an `is_explored` flag. Agents who step on cells with this flag set to `true` collects a small positive reward and negates the flag. TODO: explain why? (if not done elsewhere)

3.2.4 Visualising the Environment

To help debug the environment, a lightweight real-time visualisation is built. Integrating the visualisation directly into the training loops would lead to strong performance degradation as each RL step would have to wait for a complete render of the current state.

To decouple these systems, an independent frontend is developed with TypeScript ¬R and p5.js ¬R. On initialisation, the Rust backend spawns a background thread using the `tokio` library ¬R to manage the websocket connection setup by the `warp` library ¬R. During runtime, current state data is streamed from this websocket. Strict types are defined in both the rust simulation and frontend for the data being communicated through the websockets to ensure the system accurately reflects the current state.

To synchronise the configuration between the visualisation and the simulation backend, a Python utility extracts required parameters from the configuration files and places them into a frontend friendly format. This synchronisation is integrated into the Vite build script to ensure perfect synchronisation.

(Maybe TODO: dynamic worlds parallisd)

3.3 MARL Infrastructure

3.3.1 Actor and Critic Design

Similar to Liu et al.[1], an actor-critic network is setup following the Centralised Training Decentralised Execution (CTDE) paradigm. The PyTorch module `PPO_Critic` operates solely during the training phase and receives global knowledge of the current state of the world. The global knowledge is the complete, unmasked environmental state concatenated with the communication vectors of all agents. This is processed through a deep Multi-Layer Perceptron (MLP) body. To maintain network stability, `nn.LayerNorm` and ReLU activation functions are applied sequentially. The network outputs an estimate of the return for each agent in the given state.

The PyTorch module `PPO_Actor` controls each agent and acts only using local knowledge of the agent. This local knowledge is the agent's own state (position, inventory - what the agent currently has and needs), closest entity if within a specific radius and the communication vectors broadcasted by all other agents. This is processed through a deep MLP body, and then, as the game is POMDP, is passed through an Long Short Term Memory unit (LSTM). The LSTM allows individual agents to remember locations of stations and meeting points with other agents. The network outputs action logits (probabilities of each action to be taken) and returns the output of the LSTM for the communication protocol to use later on.

To prevent weights vanishing or exploding during early training, orthogonal initialisation was opted for rather than the default uniform initialisation PyTorch uses ¬R.

3.3.2 Memory Management

During the rollout phase, the data collected including inputs to the policy (communication values, observations) and the feedback from the simulation (rewards) must be saved for the update phase. In the vectorised environment with multiple agents over hundreds of timesteps, simply appending to a Python list is insufficient due to constant copying overhead when the list doubles in size ¬R? and memory fragmentation ¬R?. To prevent this overhead, a custom `RolloutBuffer` class is implemented. On instantiation, the buffer allocates contiguous memory for the data collected over the rollout phase in the form of a `torch.tensor` on the target hardware device. Memory allocated is for

- Local agent observations
- Global world observations
- LSTM recurrent hidden states
- Each agent's targets
- Each agent's rewards
- Broadcasted communication vectors and associated log-probabilities
- Actions chosen and associated action log-probabilities

- Critic's values
- Advantages and returns derived from Generalised Advantage Estimation (GAE).

After every environment step, data is directly inserted into the pre-allocated tensors via index-slicing. This insertion is a $O(1)$ zero-copy operation. This method minimises the data transfer across the CPU-GPU boundary which is an expensive operation.

3.3.3 Generalised Advantage Estimation

After concluding the rollout phase, the temporal difference returns and advantages need to be calculated to update the policies. To avoid unnecessary data movement, this process is executed directly on the stored tensors inside the `RolloutBuffer` instance as follows:

Algorithm 1 Compute Advantages and Returns using GAE

procedure GeneralisedAdvantageEstimation($V_{last}, \gamma, \lambda$)
  Input: Last value V_{last}, discount factor γ, GAE parameter λ
  Initialise: $gae \leftarrow 0$
  $T \leftarrow$ timesteps_per_run
  for $t = T - 1$ **down to** 0 **do**
    $r \leftarrow R[t]$ ▷ Reward at timestep t
    $v \leftarrow V[t]$ ▷ Value at timestep t from critic
    $\delta \leftarrow r + \gamma V_{last} - v$ ▷ Compute 1-step TD Error
    $gae \leftarrow \delta + \gamma\lambda \cdot gae$ ▷ Compute GAE advantage
    $A[t] \leftarrow gae$ ▷ Store advantages
    $Returns[t] \leftarrow gae + v$ ▷ Store returns
    $V_{last} \leftarrow v$ ▷ Update last value for the next iteration $(t-1)$
  end for
end procedure

As detailed in Algorithm 1, GAE starts at the final step. It calculates the one-step Temporal Difference Error (δ) - representing the immediate step reward plus the discounted expected value of the subsequent state minus the baseline value estimate of the current state. The GAE is then recursively accumulated using the decay parameter (λ) and the environmental discount factor (γ). By adding the current δ to the exponentially discounted future advantages, GAE successfully balances the bias-variance tradeoff inherent in deep reinforcement learning. The target returns for training the Critic are formulated by summing the advantage with the current state value. This algorithm has a time complexity of $O(T)$.

3.3.4 Communication Strategy Pattern

A critical part of this project is the comparative analysis of different methods of communication. This includes no communication, dynamically learned discrete symbols and AI Mother Tongue methods.

To avoid repetition of code blocks and consistency over each of these methods, an abstract base class was engineered (`Comms`) using the Python library `abc`. The specific protocols (named

`NoneComms`, `DiscreteComms`, `AimComms`) were implemented as PyTorch modules and inherited from this base class. On initialisation, the `actor` module dynamically instantiates the required protocol based on the configuration. This allowed experimental hot-swapping of protocols to be achieved seamlessly without altering the MAPPO algorithm.

3.3.5 MAPPO Algorithm

Algorithm 2 MAPPO Minibatch Update with Learned Communication

Require: Buffer minibatches, Clip hyperparameter ϵ, Value coefficient c_{vf}, Entropy coefficient c_{ent}

for batch $\in$ buffer **do**
  Extract: Observations O, O_{global}, Actions a, Comms c, Advantage A, Returns $\hat{R}$
  Extract: Old log probs $\log \pi_{\text{old}}(a), \log \pi_{\text{old}}(c)$, Initial LSTM states h_0, c_0
  ▷ **Critic Evaluation**
  $V \leftarrow V_\psi(O_{global})$ ▷ Compute new state values
  $L_{\text{critic}} \leftarrow \text{MSE}(V, \hat{R})$
  ▷ **Communication Embedding & Shifting**
  $C_{\text{cont}} \leftarrow \text{EmbedCodebooks}(c)$ ▷ Convert discrete comms to continuous embeddings
  $C_{\text{shifted}} \leftarrow \text{ShiftRight}(C_{\text{cont}}, \text{steps} = 1)$
  ▷ **Actor & Comm Protocol Forward Pass**
  $\pi_\theta(a), h' \leftarrow \text{ActorRNN}(O, h_0, c_0, C_{\text{shifted}})$
  $\pi_\phi(c), \mathcal{H}_c, L_{\text{proto}} \leftarrow \text{CommProtocol}(h', c, C_{\text{cont}}, \hat{R}, V)$
  ▷ **Action PPO Surrogate Loss**
  $r_a \leftarrow \exp(\log \pi_\theta(a) - \log \pi_{\text{old}}(a))$
  $L_{\text{action}} \leftarrow -\mathbb{E}\left[\min\left(r_a A, \text{clip}(r_a, 1-\epsilon, 1+\epsilon)A\right)\right]$
  ▷ **Communication PPO Surrogate Loss**
  $r_c \leftarrow \exp(\log \pi_\phi(c) - \log \pi_{\text{old}}(c))$
  $L_{\text{comm}} \leftarrow -\mathbb{E}\left[\min\left(r_c A, \text{clip}(r_c, 1-\epsilon, 1+\epsilon)A\right)\right]$
  ▷ **Distributions & Entropy**
  $\log \pi_\theta(a) \leftarrow \text{LogProb}(\pi_\theta, a)$
  $\mathcal{H}_a \leftarrow \text{Entropy}(\pi_\theta)$
  $L_{\text{ent}} \leftarrow -\mathbb{E}[\mathcal{H}_a + \mathcal{H}_c]$
  ▷ **Total Loss Computation**
  $L_{\text{actor}} \leftarrow L_{\text{action}} + L_{\text{comm}} + L_{\text{proto}}$
  $L_{\text{total}} \leftarrow L_{\text{actor}} + c_{vf} L_{\text{critic}} + c_{ent} L_{\text{ent}}$
  ▷ **Optimisation Step**
  $\theta, \phi, \psi \leftarrow \text{ZeroGradients}()$
  Compute $\nabla_{\theta,\phi,\psi} L_{\text{total}}$
  $\text{ClipGradNorm}(\theta, 0.5)$; $\text{ClipGradNorm}(\psi, 0.5)$
  $\text{OptimiserStep}()$
end for

Algorithm 2 depicts the update loop for MAPPO. To ensure stability in the updates, advantage normalisation is applied to the GAE advantages. ¬R

For every epoch, the data collected by the rollout phase is split into random minibatches. First, the critic is evaluated by feeding in the global observation against returns from GAE using MSE loss.

The communication vectors broadcasted in the preceding time step are collected and fed into the actor network along with the relevant local observations and LSTM hidden state. To prevent chaotic updating, the standard PPO clipped surrogate objective is used. Notably, the clipping of the communication loss and the action loss are computed separatly. This strict isolation ensures that a highly confident, but ultimately incorrect, communication update does not destructively skew the gradients of a well-performing spatial movement policy.

Each of these surrogate losses is aggregated into a single composite loss as well as an entropy loss for encouraging continued exploration. However, before applying the final gradients to the networks, the complete actor and critic parameters are clamped to a maximum L_2 norm of 0.5.

Due to imitation learning (detailed in Section 3.4.2.2), a blanket learning rate is not suitable for MAPPO. Because the actor and critic networks have already been partially optimised, aggressive learning rates risks forgetting of these pre-trained policies. To account for this, a conservative learning rate is applied to the actor's networks that were trained during imitation learning to prevent chaotic updates losing this information. The critic network is exempt from this conservative learning rate otherwise the network would be unable to rapidly adapt to the agents' shifting policies and provide accurate value function estimates. This variable learning rate is applied using `PyTorch`'s optimiser's parameter groups (`param_groups`). This careful orchestration of advantage normalisation, isolated loss clipping, gradient clamping and variable learning rates ensures monotonic policy improvement across the training landscape.

3.4 AI Mother Tongue and Emergent Communication

3.4.1 Autoencoder Architecture

One objective of this project was to investigate different Variational Autoencoders to develop this AI Mother Tongue Language. Vector-Quantised Variational Autoencoders (VQ-VAEs) and Stochastically-Quantised Variational Autoencoders (SQ-VAEs) are pure methods of autoencoders. Hierarchically-Quantised Variational Autoencoders (HQ-VAEs) are a wrapper around these pure methods. Residually-Quantised Variational Autoencoders (RQ-VAEs) are a modifiction to these pure methods. To allow for experimentation, interchangeability was key. An agnostic `Encoder` class was engineered to abstract away the shared neural network boilerplate, latent dimension mapping, and disk I/O. With this boilerplate code abstracted away, the VQ-VAE and SQ-VAE could be designed as lightweight, interchangable models, satisfying our goal.

1. **VQ-VAE**

   To map a continuous vector to a discrete vector in VQ-VAE, pairwise Euclidean distance is used. In PyTorch, this can be done using `torch.cdist` followed by the `argmin`. However, as discussed in the preparation section, argmin is non-differentiable. The Straight-Through Estimator is implemented using `quantised = x + (quantised - x).detach()`. During the forward pass, PyTorch evaluates to the discrete code. During the backward pass, PyTorch sets the gradients of the detached term to zero, allowing the gradient to flow solely through continuous latent vector x (preserving the computational graph).

   A known pathology of VQ-VAE is codebook collapse, to mitigate this in our On-Policy Communication Protocol, a dead-codebook revival heuristic is added. An exponentially

weighted moving average tracks the usage count of each code. Once a code drops below a pre-defined threshold, it is replaced by a new code randomly sampled from the current continuous input batch. This ensures usage the entire codebook remains.

2. **SQ-VAE**

   Unlike VQ-VAE which relies on the non-differentiable argmin operator, SQ-VAE models the discretisation bottleneck as a probabilistic categorical distribution. To allow backpropagation through this discrete sampling, the Gumbel-Softmax reparamatisation trick is used via PyTorch's `torch.nn.functional.gumbel_softmax`. During the forward pass, a discrete one hot-vector is sampled. During the backward pass, a differentiable vector is used. To further improve codebook utilisation, a dynamic temperature annealing scheduler is used. At the start of training, a high temperature is applied to encourage exploration of the entire codebook. As training progresses, the temperature is reduced to make the communication increasling deterministic.

3. **HQ-VAE**

   A standard issue with the previously mentioned VAEs is increasing the expressive freedom of the language requires linearly growing the codebook size which becomes computationally intensive (computing pairwise difference between every embedding). To solve this issue, a HQ-VAE is implemented.

   Instead of utilising one large codebook, it is split into two. Initially the environment observations are passed through a shared MLP. A `top_quantiser` projects these features into a continuous latent space which is quantised using the top-level codebook. These codes are for communicating macro-level strategies.

   For micro-level strategies, `bottom_quantiser` projects the envionment observations concatenated with the top-level quantised code to another continuous latent space which is quantised using the bottom-level codebook. By forcing this conditional hierarchy, the language has an effective voacbulary size of $|\text{top_level_codebook}| * |\text{bot_level_codebook}|$ while being much less computationally intensive.

3.4.2 Offline Pre-training

Training VAEs

The AI Mother Tongue framework requires these autoencoders to be trained before the RL phase of the learning process.

Data for VAE training is collected from an expert agent who plays the game. This expert agent is granted access to an oracle which houses the unmasked, global observation state. The expert agent then employs a greedy heuristic, taking steps directly to the nearest required station until interaction is possible. By deploying this greedy expert agent, a dataset can be collected with high quality spatial movements.

A significant challenge with training autoencoders is codebook collapse. To mitigate this, the codebooks are not randomly initialised, instead data-driven codebook initialisation is used to set the spatial priors of the codebook.

A subset of the data collected from the expert agent is used to find the spatial priors by passing the data through the initialised, but untrained, encoder. Centroids are then found by using

Algorithm 3 Data-driven Codebook Initialization via K-Means

procedure FitRQStream(X, Q, d)
  Input: Initial continuous latents/residuals X, Quantiser Q, latent dim d
  $X \leftarrow \text{Reshape}(X, [-1, d])$ ▷ Flatten batch and time dimensions
  $L \leftarrow Q.\text{rq_levels}$
  $K \leftarrow \text{VocabSize}$
  for $l = 1$ **to** L **do**
    $X_{\text{norm}} \leftarrow \frac{X}{||X||_2+\epsilon}$ ▷ L2 Normalise residuals ($\epsilon = 10^{-5}$)
    $C \leftarrow \text{MiniBatchKMeans}(X_{\text{norm}}, \text{clusters} = K)$ ▷ Find K centroids
    if Encoder is SQ-VAE **then**
      $C \leftarrow \frac{C}{||C||_2}$ ▷ Normalise centroids onto unit sphere
    end if
    $Q.\text{codebook}[l] \leftarrow C$ ▷ Set embedding weights
    $A \leftarrow \text{PredictClusters}(X_{\text{norm}}, C)$ ▷ Get cluster assignments
    $X \leftarrow X - C[A]$ ▷ Compute residuals for the next RQ level
  end for
end procedure

procedure SetPriors($M, \mathcal{D}$)
  Input: Model M, Dataloader $\mathcal{D}$
  $\mathcal{Z}_{\text{top}} \leftarrow [], \quad \mathcal{Z}_{\text{bot}} \leftarrow [], \quad \mathcal{Z} \leftarrow []$
  ▷ **Step 1: Collect Continuous Latents**
  for batch $O \in \mathcal{D}$ **do**
    $z \leftarrow M.\text{encoder.GetContinuous}(O)$
    Append z to $\mathcal{Z}$
  end for
  ▷ **Step 2: Fit Residual Quantisation Streams**
  $X \leftarrow \text{Concatenate}(\mathcal{Z})$
  FitRQStream($X, M.Q, d$)
end procedure

the `MiniBatchKMeans` function from the Python library `scikit-learn` for K clusters (where K is the vocabulary size). If the current VAE is of type vMF SQ-VAE then we must also normalise the embeddings onto a unit sphere with `torch.nn.functional.normalize(..., p=2.0)`. These centroids overwrite the randomly initialised embeddings in the codebook. For HQ-VAEs and RQ-VAEs (more than one codebook), the clustering operates sequentially. Once the top-level codebook is complete, the residuals are calculated for the data and the next codebook is populated using the same method. This continues until all codebook embeddings have been overwritten with these priors.

During the offline optimisation loop, applying MSE loss across the entire reconstructed vector proved suboptimal due to the heterogeneous nature of the observations. The majority of the data is binary flags, one-hot vectors and the remaining data consists of coordinates between $[0, 1]$. Sliced Composite Loss can be applied to maximise the effectiveness of the reconstruction loss. For the binary values such as flags and one-hot vectors, a Binary Cross-Entropy (BCE) loss function can be used. For the coordinates, MSE loss can be used. A multiplier is added to the MSE loss to improve the accuracy of the reconstruction of the coordinates.

After learning the VAE, it is loaded into the encoder for the RL phase and the parameters are frozen to prevent further training. This is done in PyTorch via `parameter.requires_grad = False`.

Imitation Learning

Due to problems such as the non-stationary problem and credit assignment problem, MARL takes a prohibitive number of training steps to converge on a viable policy. To reduce the amount of steps, Liu et al.[1] suggests implementing imitation learning to pre-train the actor and critic networks.

Similar to training the VAEs, a dataset must be collected from the expert agent. However, two significant issues arise. The expert agent bases its actions solely from an oracle who tells the expert agent where their next target is. The expert agent also does not use any of the discrete codebook or any communication for that matter. To fix both these issues, a spatial hash is applied to the arena. The arena is broken into K discrete areas, where K is the vocabulary size. A unique id can be assigned to each of these areas:

$$\text{unique_id} = \text{ROW_SIZE} * y + x$$

where x and y are the discrete column and row indices, respectively.

A lookup in the codebook using *unique_id* is done and the result is injected into the communication channels during training.

Now that the data is collected, the networks are optimised via Behavioural Cloning. The actor network is trained using the Cross-Entropy loss (using `torch.nn.functional.cross_entropy`) to help the actor learn basic movement. The critic network is trained using the Mean-Squared Error loss (using `torch.nn.functional.mse_loss`) to help the critic predict the expert's expected temporal difference returns.

On completion, these models are saved to disk and loaded when the reinforcement learning begins.

3.4.3 Reflection Strategies

In the AI Mother Tongue method, the VAE is frozen during the RL phase. This means the agents must learn to use the existing language rather than manipulating the codebook to improve their coordination. Liu et al. [1] propose using reflection strategies to teach the agent semantic meanings of the codes. These strategies are implemented by adding small MLP heads onto the actor output and allowing the auxiliary heads' losses to flow into the actor to further adjust the weights.

Two different inputs to the auxiliary heads exist.

1. *sender_context* defined as the concatenation of the output of the actor's recurrent unit and the communication vector that actor broadcasted.
2. *receiver_context* defined as the concatenation of the output of the actor's recurrent unit and all other agents' communication vectors.

Reflection strategies are defined as follows:

1. **Contextual Value Learning** This reflection strategy requires the agent to understand the utilitarian value of its message. To implement this, an auxiliary head named `predict_agents_returns` is defined. The *sender_context* is taken as input and optimises to predict the return calculated from GAE using MSE loss.

2. **Theory of Mind** Theory of mind is achieved once agents can infer the current goals of agents from their communication vectors. This is achieved through two auxiliary losses:

   - **Sender Goal Alignment** The sender needs to understand its own intent. An auxiliary head named `predict_intent_sender_goal` is defined. The *sender_context* is taken as input and optimises to predict the coordinates of the next target (decided by the same logic as the expert agent) using MSE loss.
   - **Receiver Goal Alignment** The sender needs to understand other agents' intent from their communication vectors. An auxiliary head named `predict_intent_receiver_goal` is defined. The *receiver_context* is taken as input and optimises to predict the coordinates of all other agents' next target using MSE loss. To filter out the agent's own communication vector, an inverse identity mask (`~torch.eye`) is instantiated on the target hardware device to filter out the agent's self-broadcasted communications

These losses are all aggregated into one and multiplied by a pre-defined coefficient (eg $\lambda = 0.01$). This loss is returned and added to the composite MAPPO objective function for backpropagation.

3.4.4 Architectural Divergence: On-Policy VQ-VAE

Unlike AI Mother Tongue protocol, this

3.5 Software Engineering, Reproducibility, and Observability

MARL systems takes thousands of rollout phases to train. Leaving the system until the end of training to evaluate would render debugging practically impossible. To ensure strict scientific rigor, custom observability and checkpointing frameworks were implemented.

3.5.1 Real-time System Monitoring

To prevent computational bottlenecks, three lightweight classes were engineered:

- `MetricTracker` This object collects any statistics across the program. Instances can then be passed to the following classes for saving and displaying stastics.
- `Logging` This constantly flushes the statistics collected in `MeticTracker` instances to disk to keep run data even in the case of crashes.

- `Visualiser` Designed specifically for the RL phase, this class uses the Python library `rich` to display relevant statisitics to that run. The visualisation displays hyperparameters from the configuration files, the current state of the RL loop (rollout or update phase) and shows the past X training steps of `MetricTracker` statistics in the form of a basic graph.

`MetricTracker` and `Logging` are usable in the RL loop, imitation learning and VAE training.

3.5.2 Configuration Management

A modular configuration system was engineered to manage the large number of hyperparamaters. Separate configuration files were developed for:

- Actor Hyperparamaters
- Critic Hyperparamaters
- Imitation Learning Configuration
- VAE Training Configuration
- RL Training Configuration
- Environment Configuration
- Communication Protocol Configuration
- MAPPO Hyperparamaters

To prevent silent errors, the Python library `pydantic` was used. The configuration dataclass inherited from `pydantic.BaseModel` which defined the expected schema for each of the aformentioned files. This dataclass is then passed throughout the program.

3.5.3 Checkpointing

Simply storing the logs accumulated over the run is insufficient for resuming of training incase of hardware failure. For this functionaliy, a `CheckpointManager` class is developed. On instantiation, a config file is saved to a new folder for future reference. Then every X training steps (eg $X = 100$, but configurable in the configuration files), the actor, critic models and their respective optimisers are saved. These saved models and optimisers can be loaded on start to reproduce any results.

3.6 Summary

Chapter 4

Evaluation

Compelling argument demonstrating success criteria met, or well-justified explanation of different direction taken. Excellent critical thought and interpretation which substantiate any claims of success.

4.1 Evaluation Setup

4.2 System Performance

Reinforcement Learning with multiple agents requires millions of datapoints for updating the polcies and an equal number to evaluate the language and policy. This section quantitivley measures the performance gains through the custom systems developed. Specifically, it analyses the steps per second of the rust simulation developed and the memory efficiency of the `RolloutBuffer`. Benchmarks were evaludated on NVIDIA RTX 5060 Ti and AMD Ryzen 7 3700X.

4.2.1 Vectorised Environment Throughput

A major issue with Python running multiple threads concurrently is the GIL. While Python's builtin library `multiprocessing` can, in theory, bypass this limitation, it introduces considerable overhead. For this RL project, all threads require synchronisation at every simulation timestep.

To justify developing the simulation in Rust rather than Python, a simple mock simulation was developed in Python. This mock simulation was compared to the vectorised Rust backend. Environment steps per second (SPS) was calculated to analyse the throughput of each of the simulations. Number of worlds parallised was scaled ($W \in \{2^0, 2^1, ..., 2^8\}$) to measure concurrency efficiency.

As shown in figure 4.1, the Python baseline completely fails to match the Rust simulation. Despite being able to bypass the GIL and using multiple CPU cores, the serialisation and the Inter-Process Communication overhead required to transfer the necassary data between isolated processes at every time step heavily bottleneck the throughput.

In contrast, the Rust is able utilise `Rayon`'s thread-pool to efficiently manage the parallel world steps nativly in C-space. After a minor scheduling overhead at $W = 2$, throughput scales linearly with number of worlds parallised in $\log_2$. By using Rust's native Rayon threading, the sequential fraction of the program is minimised per Amdahl's Law allowing near-perfect scaling that saturates the CPU cores.

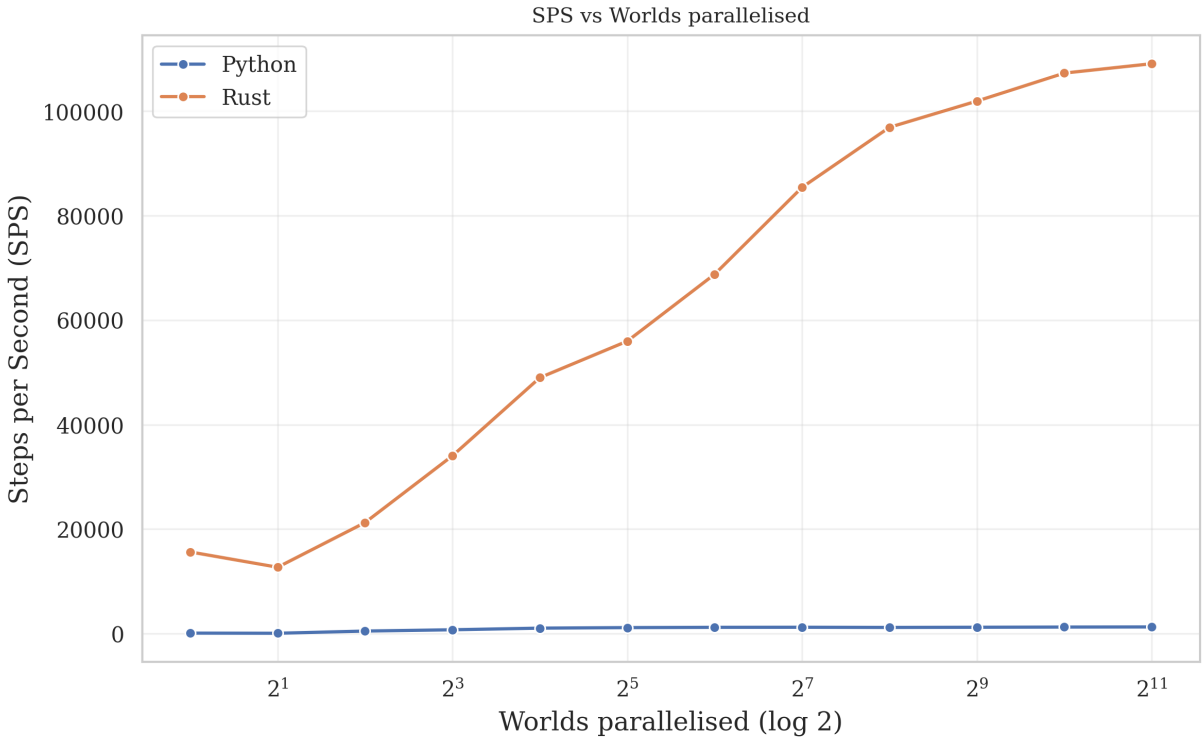

Figure 4.1: Concurrent Environment Throughput Performance. Comparison of Python multiprocessing and Rust with Rayon across an exponentially increasing number of parallel worlds

4.2.2 Rollout Buffer Efficiency

A secondary major potential bottleneck is transferring the results of the rollout phase to the training phase (across CPU-GPU boundary). To evaluate the `RolloutBuffer`, the memory allocation and transfer overhead is compared to a baseline method (appending to Python lists and executing `torch.stack` after the rollout phase).

Memory Strategy	Allocation Time (s)	Peak VRAM Overhead (MB)
Dynamic Python Lists	6.47 ± 0.18	1213.45
Pre-allocated RolloutBuffer	6.79 ± 0.23	598.23

Table 4.1: Memory Management Overhead per Epoch: $T = 20,000$, $W = 32$

As shown in 4.1, the `RolloutBuffer` method reduces VRAM by over 50%. This is as Python's dynamic arrays often require copying their contents from one memory location to another to increase their size. Whilst there is a 5% improvement in allocation time when using Python's dynamic list, the large improvement in VRAM will enable scaling this method to millions of datapoints.

4.3 Offline Pre-training

4.3.1 Reconstruction and Commitment Quality

4.3.2 Initial Codebook Health

4.4 Online MARL Task Performance

4.4.1 Baseline Comparisons

4.4.2 Bandwidth-Performance Trade-off

4.5 Analysis of Emergent Linguistics

4.5.1 Spontaneous Semantic Compression

4.5.2 Intent and Semantic Mapping

4.5.3 Decoding the HQ-VAE

4.6 Architectural Ablation Studies

4.6.1 Reflection Strategies - Are They Needed?

4.6.2 VQ-VAE vs SQ-VAE

Chapter 5

Conclusions

Conclusions provide a clear and thorough summary, with interesting future work suggested. Thoughtful personal reflection on the lessons learned.

5.1 Achievements

5.2 Lessons Learnt

5.3 Future Work

Bibliography

[1] Hung Ming Liu. "AI Mother Tongue: Self-Emergent Communication in MARL via Endogenous Symbol Systems". In: *arXiv preprint arXiv:2507.10566* (2025). URL: https://arxiv.org/abs/2507.10566.

[2] Ronald J. Williams. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning". In: *Machine Learning* 8 (1992), pp. 229–256. DOI: 10.1007/BF00992696.

[3] Chunxu Yu et al. "The Surprising Effectiveness of MAPPO in Cooperative Multi-Agent Games". In: *arXiv preprint arXiv:2301.08454* (2023).

[4] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. eprint: arXiv:1707.06347.

[5] Shivang Shrivastav. *Generalized Advantage Estimation (GAE): A Deep Dive into Bias, Variance, and Policy Gradients*. Medium. Accessed: 2026-04-02. Mar. 2025. URL: https://shivang-ahd.medium.com/generalized-advantage-estimation-a-deep-dive-into-bias-variance-and-policy-gradients-a5e0b3454dad.

[6] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. *Neural Discrete Representation Learning*. 2017. eprint: arXiv:1711.00937.

[7] Yuhta Takida et al. *SQ-VAE: Variational Bayes on Discrete Representation with Self-annealed Stochastic Quantization*. 2022. eprint: arXiv:2205.07547.

[8] Yuhta Takida et al. *HQ-VAE: Hierarchical Discrete Representation Learning with Variational Bayes*. 2023. eprint: arXiv:2401.00365.

[9] PyO3 Project. *PyO3 - Rust bindings for the Python interpreter*. https://github.com/PyO3/pyo3. 2017. (Visited on 04/10/2026).

[10] PyO3 Project. *setuptools-rust*. https://pypi.org/project/setuptools-rust/. 2021. (Visited on 04/10/2026).

[11] Refactoring Guru. *Adapter*. Refactoring.Guru. 2024. URL: https://refactoring.guru/design-patterns/adapter (visited on 04/10/2026).

[12] NumPy Developers. *NumPy*. https://numpy.org. Version 2.4. Dec. 2025.

Appendix A

Proposal